

E-Commerce Microservices Architecture Package

Software Architecture • ERD • Software Design • User Stories

v0.1 — Baseline design from supplied project specification

Purpose: Turn the supplied project concept into a coherent, implementation-ready documentation baseline. The source defines the core technology stack and four main business/technical services; this document adds explicit design decisions only where required and labels them as PROPOSED.

Status: Architecture baseline / pre-implementation

Primary implementation language: Java 21 + Spring Boot 3.x

Document Map

Section	Purpose	Status
1. Source Baseline	What is explicitly required by the supplied specification.	Source-derived
2. Software Architecture	System context, components, communication, deployment and quality attributes.	Baseline + proposed details
3. ERD	Service-owned data model and logical cross-service references.	Baseline + proposed details
4. Software Design	Internal service structure, APIs, events, state machine and conventions.	Proposed implementation design
5. User Stories	Business and technical stories with acceptance criteria.	Proposed product backlog
6. Decision Log	Architecture decisions and scope boundaries for v0.1.	Proposed / to be validated

Reading rule: “SOURCE” means the point is explicitly supported by the provided project plan. “PROPOSED” means it is an architecture/design decision introduced here to make implementation concrete. Proposed items can be changed without contradicting the original source.

1. Source Baseline

The supplied plan defines a modern portfolio-grade e-commerce microservice system focused on distributed-system engineering rather than a simple CRUD showcase. The explicit baseline is:

- Java 21, Spring Boot 3.x, Java Records for DTO mapping, and virtual threads enabled through `spring.threads.virtual.enabled=true`.
- Spring Cloud Gateway as the single entry point, with Keycloak using OAuth 2.0 / OpenID Connect and JWT-based access control. Gateway validates access tokens; final RBAC checks remain in services.
- Micrometer Tracing + OpenTelemetry, Spring Boot Actuator metrics in Prometheus format, and distributed traces exported to Zipkin or Jaeger.
- A docker-compose.yml that starts the application infrastructure as one local environment.
- Product / Catalog Service with PostgreSQL, Flyway, Spring Data JPA, @EntityGraph to avoid N+1, Redis Cache-Aside, Redisson RLock against cache stampede, and Spring Batch for supplier CSV bulk imports.
- Order Service with an order state machine and Transactional Outbox. Order and outbox record are written in one PostgreSQL transaction; Debezium reads the WAL and publishes OrderCreatedEvent to Kafka.
- Payment Service as a Kafka consumer, with idempotent processing using a local `processed_events` table, plus a mock external payment integration through Spring RestClient.

- Audit / Notification Service that consumes business events, stores unstructured audit data in MongoDB, and sends e-mail / push notifications.

1.1 Scope Gaps That Must Be Designed

Gap	Status in source	v0.1 treatment
Exact product/order/payment table columns	Not specified	PROPOSED in ERD
Order statuses and transitions	State machine required; values not specified	PROPOSED minimal payment-focused state machine
Exact REST API endpoints	Not specified	PROPOSED API surface
Kafka topic names and event envelopes	Only OrderCreatedEvent is named	PROPOSED versioned event contract
Inventory, shipping, cart, search services	Not specified	Explicitly out of v0.1 scope
Notification persistence model	Not specified	Deferred; Mongo audit store is source-backed

2. Software Architecture

2.1 Architecture Goals

- Demonstrate production-style boundaries and failure handling while remaining buildable by one developer.
- Keep each service independently deployable and the owner of its own persistence model.
- Avoid distributed dual writes by publishing order events through Transactional Outbox + Debezium CDC.
- Make repeated Kafka delivery safe through idempotent consumers.
- Keep reads fast through Redis while preventing cache stampede under concurrent misses.
- Make failures diagnosable through consistent metrics, traces and correlation IDs.

2.2 System Context and Containers

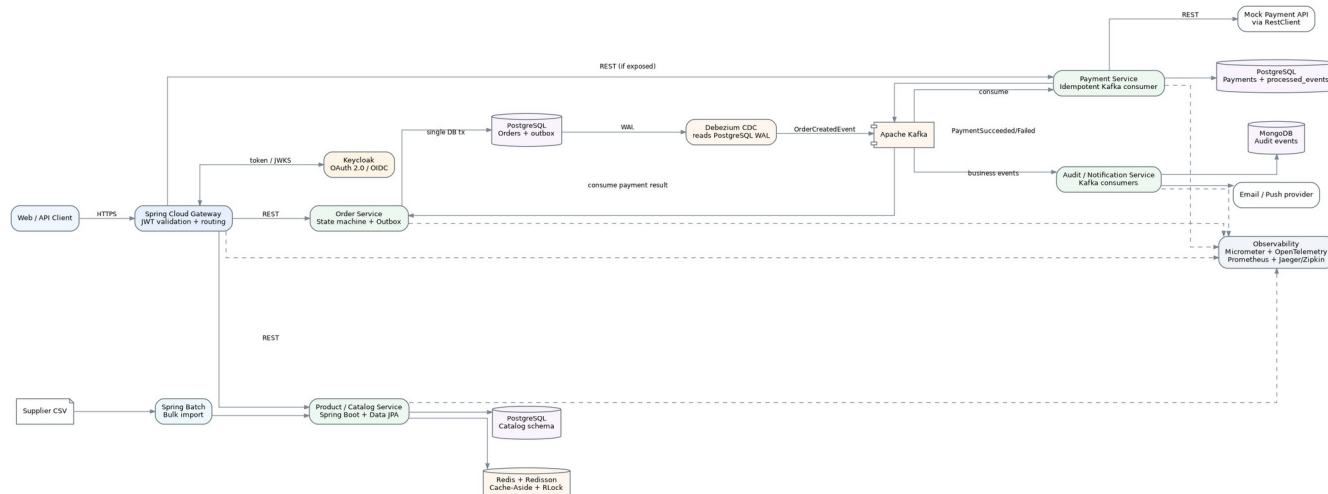


Figure 1. Proposed container-level architecture. Solid arrows represent business/data flow; dashed arrows represent observability signals.

2.3 Service Responsibilities

Component	Primary responsibility	Owned data / infrastructure
API Gateway	Entry point, routing, token validation, cross-cutting request logging.	No business database
Keycloak	Identity provider, authentication, OAuth2/OIDC token issuance.	Keycloak-owned identity store
Catalog Service	Products, categories, attributes, read optimization, supplier imports.	Catalog PostgreSQL schema + Redis cache
Order Service	Order lifecycle and state machine; writes Outbox events atomically with orders.	Order PostgreSQL schema + outbox table
Payment Service	Consumes order events, performs payment attempt, guarantees idempotent event processing.	Payment PostgreSQL schema + processed_events
Audit / Notification	Consumes business events, writes audit history, triggers customer notifications.	MongoDB audit documents
Kafka + Debezium	Durable asynchronous event transport; CDC bridge from outbox to Kafka.	Kafka topics + connector offsets
Observability stack	Metrics, traces and health data for debugging and operations.	Prometheus + Jaeger or Zipkin

2.4 Communication Model

Flow	Mode	Reason
Client → Gateway → Catalog / Order	Synchronous REST	User-facing request/response path.
Order DB → Debezium → Kafka	Asynchronous CDC	Eliminates application-level dual-write between DB and Kafka.
Kafka → Payment	Asynchronous event consumption	Payment can retry independently and tolerate temporary unavailability.
Payment → Kafka → Order	PROPOSED asynchronous result event	Keeps payment result handling decoupled; source allows sync or async.
Kafka → Audit / Notification	Asynchronous fan-out	Audit and notification must not block core checkout flow.
Payment → mock provider	Synchronous RestClient	Source explicitly suggests external API simulation through RestClient.

2.5 Data Ownership Rules

- Each service is the only writer to its own data store.
- No physical foreign keys cross service boundaries.
- Cross-service identifiers (product_id, order_id, user_id) are logical references only.
- Order items should snapshot commercial data such as product name and unit price so historical orders do not change when the catalog changes. [PROPOSED]
- Keycloak remains the source of truth for authentication identity; business services store only the subject/user identifier they require. [PROPOSED]

2.6 Security Model

- Keycloak issues OAuth 2.0 / OIDC access tokens.
- Spring Cloud Gateway rejects missing/invalid JWTs before protected routes are forwarded.
- Each microservice also acts as a Resource Server and enforces endpoint-level RBAC. This prevents “gateway-only security” from becoming a single bypass point. [Architecture interpretation of source]
- Suggested roles for v0.1: CUSTOMER, CATALOG_ADMIN, OPS_ADMIN. [PROPOSED]

2.7 Reliability Patterns

Risk	Pattern	Implementation
DB update succeeds but Kafka publish fails	Transactional Outbox	Order + outbox event in same PostgreSQL transaction; Debezium publishes from WAL.

Kafka redelivers an event	Idempotent consumer	Payment checks processed_events by event_id before payment side effects.
Many concurrent cache misses for same key	Cache stampede	Redisson RLock around cache population / invalidation path.
Slow audit/notification dependency	Async fan-out	Audit/notification consumes Kafka independently from checkout.
Hard-to-debug distributed failures	Tracing + metrics	Trace ID propagated; Prometheus metrics and Jaeger/Zipkin traces.

2.8 Deployment Topology for Development

- docker-compose.yml starts PostgreSQL instances/schemas, Redis, Kafka, Debezium Connect, MongoDB, Keycloak, Prometheus and Jaeger/Zipkin.
- Application services can initially run from the IDE against the compose infrastructure; later they can also be containerized. [PROPOSED workflow]
- Every service exposes /actuator/health and /actuator/prometheus. [PROPOSED concrete endpoint choice consistent with source]

2.9 Non-Goals for v0.1

- No Inventory Service, Shipping Service, Cart Service, recommendation engine or Elasticsearch/OpenSearch unless added in a later milestone.
- No real bank/card processing. Payment is a controlled mock integration.
- No Kubernetes requirement in the first milestone; Docker Compose is the explicit source requirement.
- No shared “common database” between microservices.

3. ERD — Entity Relationship Design

Important: This ERD intentionally separates service-owned stores. Dashed relationships are logical references only and must never become cross-service foreign keys. Entities marked PROPOSED are required to make the source architecture implementable but are not explicitly named in the source.

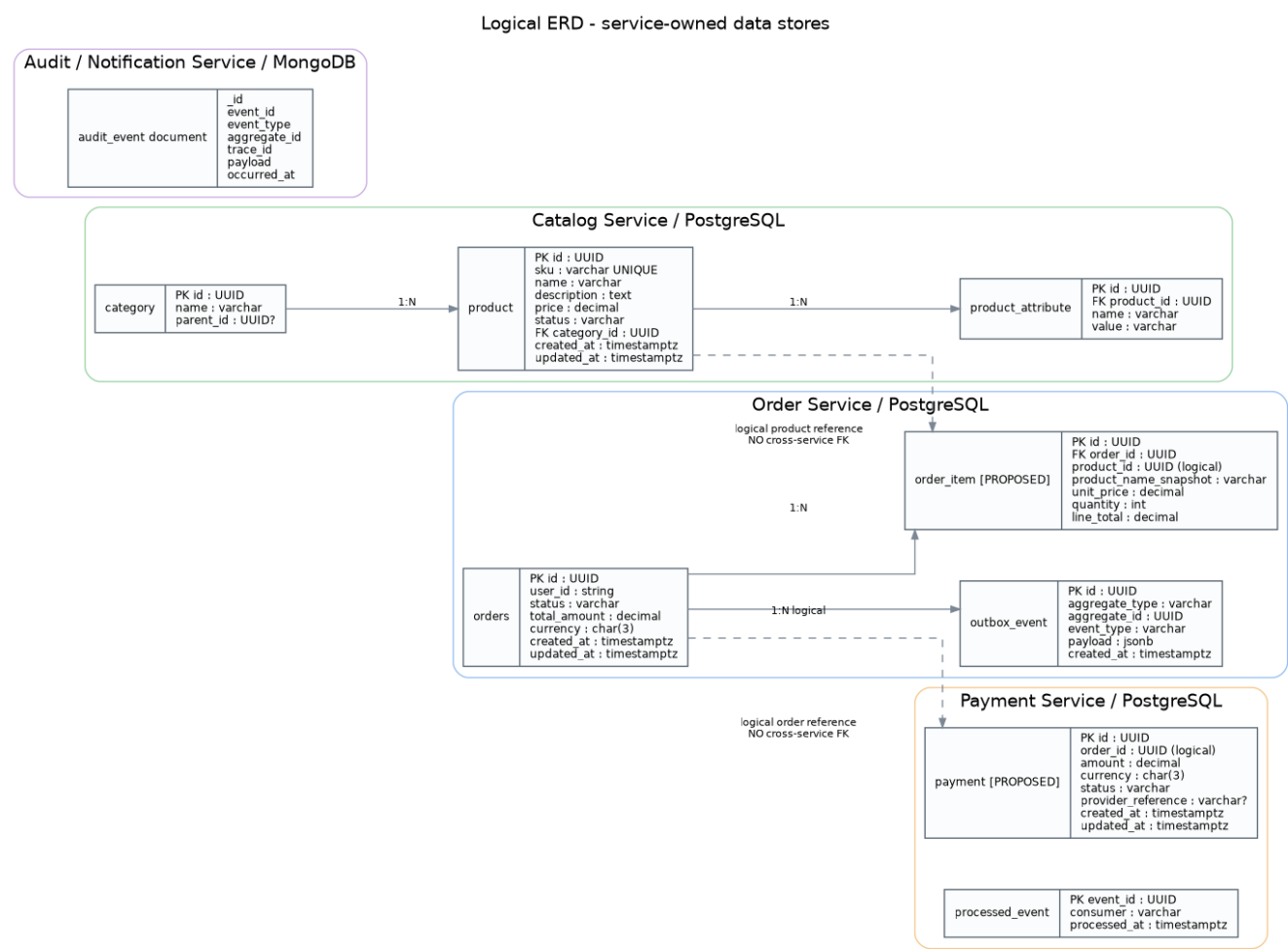


Figure 2. Logical ERD across service-owned data stores.

3.1 Catalog Service Tables

Entity	Key fields	Notes
category	id, name, parent_id	parent_id enables optional category hierarchy. [PROPOSED]
product	id, sku, name, description, price, status, category_id, timestamps	Core catalog entity. Category/attribute relationships are source-supported; exact columns are proposed.
product_attribute	id, product_id, name, value	Flexible per-product attributes;

		loaded with products via EntityGraph where needed.
--	--	--

3.2 Order Service Tables

Entity	Key fields	Notes
orders	id, user_id, status, total_amount, currency, timestamps	State machine aggregate root.
order_item [PROPOSED]	id, order_id, product_id, product_name_snapshot, unit_price, quantity, line_total	Required for a meaningful e-commerce order; snapshots avoid historical drift.
outbox_event	id, aggregate_type, aggregate_id, event_type, payload, created_at	Written in same DB transaction as order mutation; Debezium reads changes from WAL.

3.3 Payment Service Tables

Entity	Key fields	Notes
payment [PROPOSED]	id, order_id, amount, currency, status, provider_reference, timestamps	Persists payment attempt/result for traceability.
processed_event	event_id, consumer, processed_at	Source-required idempotency ledger. event_id is unique.

3.4 Audit / Notification Store

Document	Fields	Notes
audit_event	_id, event_id, event_type, aggregate_id, trace_id, payload, occurred_at	MongoDB document; flexible structure is aligned with unstructured audit-log requirement.
notification record	TBD	Source requires e-mail/push behavior but does not define persistence; decide only if retries/history require it.

4. Software Design

4.1 Repository / Module Strategy

PROPOSED: Start as a monorepo for portfolio clarity while preserving independent service build

boundaries. Each service has its own Spring Boot application, Flyway migrations, Dockerfile and tests. Shared code should be limited to event-contract artifacts or duplicated intentionally to avoid tight runtime coupling.

Module	Example path
gateway-service	services/gateway-service
catalog-service	services/catalog-service
order-service	services/order-service
payment-service	services/payment-service
audit-notification-service	services/audit-notification-service
event-contracts (optional)	libs/event-contracts
infrastructure	infra/docker-compose.yml, Kafka/Debezium/Keycloak config
docs	docs/architecture, docs/erd, docs/api, docs/adr

4.2 Internal Service Package Design

Recommended structure inside each service [PROPOSED]:

- api — REST controllers, request/response records, validation.
- application — use-case services, transaction boundaries, orchestration.
- domain — entities/value objects, domain rules, state transitions.
- infrastructure — JPA repositories, Kafka adapters, Redis adapters, RestClient, persistence configuration.
- config — security, observability, serialization and Spring configuration.

4.3 Proposed REST API Surface

Service	Method / Path	Purpose
Catalog	GET /api/v1/products	Search/list products.
Catalog	GET /api/v1/products/{id}	Read product detail; cache candidate.
Catalog Admin	POST /api/v1/admin/products/import	Trigger or submit supplier CSV bulk import.
Order	POST /api/v1/orders	Create order and atomically write outbox event.
Order	GET /api/v1/orders/{id}	Read order status.
Order	GET /api/v1/orders	List current customer orders.
Payment	No public endpoint required in	Primary flow is Kafka-driven;

	v0.1	admin/debug endpoint can remain profile-restricted.
--	------	---

4.4 Order State Machine

PROPOSED minimal state model: Because inventory and shipping are outside current scope, keep the order lifecycle focused on checkout/payment instead of inventing fulfillment states.

From	Event	To	Rule
CREATED	OrderCreated	PAYMENT_PENDING	Order persisted and outbox event created.
PAYMENT_PENDING	PaymentSucceeded	PAID	Payment result accepted exactly once.
PAYMENT_PENDING	PaymentFailed	PAYMENT_FAILED	Failure is recorded and surfaced to customer.
CREATED / PAYMENT_PENDING	CancelOrder	CANCELLED	Allowed only before PAID.

4.5 Event Contracts

Use a versioned event envelope [PROPOSED] so tracing, deduplication and evolution are explicit:

```
{
  "eventId": "uuid",
  "eventType": "OrderCreated",
  "eventVersion": 1,
  "aggregateId": "order-uuid",
  "occurredAt": "2026-10-02T...Z",
  "traceId": "...",
  "data": { ... }
}
```

Event	Producer	Consumers	Source status
OrderCreatedEvent	Order Service via Outbox/Debezium	Payment, Audit/Notification	Explicitly named in source
PaymentSucceededEvent	Payment Service	Order, Audit/Notification	PROPOSED async choice
PaymentFailedEvent	Payment Service	Order, Audit/Notification	PROPOSED async choice
ProductImportedEvent	Catalog Service	Audit/Notification	Optional / later

4.6 Payment Idempotency Algorithm

1. Kafka consumer receives an event carrying eventId.
2. Begin a local database transaction.
3. Check processed_event for eventId + consumer name.
4. If present, commit/return without executing payment side effects.
5. If absent, perform the controlled payment operation and persist payment result.
6. Insert processed_event record before committing.
7. Publish payment result event. For stronger delivery guarantees, a Payment Outbox can be added as a follow-up architecture decision. [PROPOSED improvement]

4.7 Catalog Cache Design

- Read-through behavior follows Cache-Aside: service checks Redis, falls back to PostgreSQL, then populates Redis.
- Popular product keys should have a bounded TTL. Exact TTL is configuration, not hard-coded architecture.
- On concurrent cache miss/rebuild, Redisson RLock protects one loader per product/cache key.
- Write operations invalidate or refresh affected cache keys after the database transaction succeeds.

4.8 Error Handling and API Contract

- Use a consistent error object containing timestamp, traceId, code, message, and validation details. [PROPOSED]
- Do not expose internal stack traces to clients.
- Map domain conflicts (for example invalid order transition) to 409 Conflict; validation to 400; missing resources to 404. [PROPOSED]
- Trace ID should be returned in responses to correlate user-facing errors with distributed traces. [PROPOSED]

4.9 Testing Strategy

Layer	What to test	Suggested tools
Unit	State transitions, pricing snapshots, idempotency decisions, cache loader logic.	JUnit 5, Mockito
Slice	JPA mappings, controllers, security rules.	@DataJpaTest, MockMvc/WebTestClient
Integration	PostgreSQL, Kafka, Redis, Mongo behavior.	Testcontainers
Contract	Event schema compatibility and REST DTO expectations.	JSON schema / Spring Cloud Contract (optional)
End-to-end	Create order → Kafka → payment → order result → audit.	Docker Compose test environment

5. User Stories

Status: The source is architecture-focused and does not provide an explicit product backlog. The stories below are therefore PROPOSED and trace directly to the required services and engineering behaviors.

US-01 — Browse product catalog

Persona: Customer

As a customer, I want to browse products so that I can choose what to order.

Acceptance criteria

- Given products exist, when I request the catalog, then I receive paginated product summaries.
- Product reads may be served from Redis without changing the API response.
- Only active/available catalog items are shown according to the product status rule defined later.

US-02 — View product details

Persona: Customer

As a customer, I want to open a product and see its category and attributes.

Acceptance criteria

- Product, category and attributes are retrieved without an N+1 query pattern.
- Repeated popular reads can be served from Cache-Aside Redis.
- Concurrent cache rebuilds do not stampede the database.

US-03 — Create an order

Persona: Customer

As a customer, I want to create an order from selected products so that payment can begin.

Acceptance criteria

- Order and items are persisted in one Order Service transaction.
- An outbox record for OrderCreatedEvent is persisted in that same transaction.
- The API returns an order identifier and initial status.

US-04 — Publish order event reliably

Persona: Platform

As the platform, I want committed orders to be published to Kafka without a dual-write race.

Acceptance criteria

- Debezium reads the Order DB WAL and publishes the outbox change.

- If Kafka is temporarily unavailable, the committed order remains in PostgreSQL and can be published when the pipeline recovers.
- Order creation does not directly depend on a successful Kafka network call inside the DB transaction.

US-05 — Pay for an order

Persona: Customer

As a customer, I want my created order to be processed for payment.

Acceptance criteria

- Payment Service consumes OrderCreatedEvent.
- A mock external payment API is called through RestClient.
- Payment result is persisted and a success/failure result is communicated back to Order Service.

US-06 — Prevent duplicate payment

Persona: Platform

As the platform, I want duplicate Kafka deliveries to be harmless so that the customer is not charged twice.

Acceptance criteria

- Every consumed event has a unique eventId.
- Payment Service checks processed_events before performing side effects.
- A previously processed event is acknowledged/skipped without repeating the payment operation.

US-07 — See payment status

Persona: Customer

As a customer, I want the order status to reflect the payment outcome.

Acceptance criteria

- PaymentSucceeded transitions PAYMENT_PENDING → PAID.
- PaymentFailed transitions PAYMENT_PENDING → PAYMENT_FAILED.
- Invalid state transitions are rejected and observable.

US-08 — Bulk import supplier catalog

Persona: Catalog Administrator

As a catalog administrator, I want to import a large CSV supplier file so that the catalog can be updated efficiently.

Acceptance criteria

- Import is implemented with Spring Batch.
- Invalid rows are reported with a deterministic error strategy.
- Large imports do not require loading the whole file into memory.

US-09 — Control access by role

Persona: Security Administrator

As an administrator, I want protected actions to require the correct role.

Acceptance criteria

- Unauthenticated requests to protected routes are rejected.
- Gateway validates JWT access tokens.
- Each downstream service independently checks the role needed for its operation.

US-10 — Trace a request end to end

Persona: Operations

As an operator, I want one trace identifier across services so that distributed failures can be diagnosed.

Acceptance criteria

- Trace context propagates through synchronous calls and Kafka messages where supported by the selected instrumentation.
- Metrics are exposed for Prometheus.
- Traces are visible in Jaeger or Zipkin.

US-11 — Record business events

Persona: Audit / Notification

As the audit subsystem, I want to persist business events so that order/payment activity can be investigated later.

Acceptance criteria

- Audit service subscribes to relevant Kafka topics.
- Event data is stored in MongoDB with eventId, event type and traceId where available.
- Reprocessing policy prevents accidental uncontrolled duplication in the audit store. [PROPOSED detail]

US-12 — Receive order/payment notification

Persona: Customer

As a customer, I want to receive an e-mail or push notification when important order/payment events occur.

Acceptance criteria

- Notification handling is asynchronous from the core order request.
- A notification failure does not roll back a paid order.
- Retry/dead-letter behavior will be defined in a later reliability ADR. [PROPOSED follow-up]

5.1 Story-to-Architecture Traceability

Story	Primary service/pattern
US-01, US-02	Catalog Service, JPA EntityGraph, Redis Cache-Aside, Redisson
US-03, US-04	Order Service, PostgreSQL, Transactional Outbox, Debezium, Kafka
US-05, US-06	Payment Service, RestClient, processed_events idempotency
US-07	Order state machine + payment result events
US-08	Spring Batch + Catalog PostgreSQL
US-09	Keycloak + Gateway + service-level RBAC
US-10	Micrometer, OpenTelemetry, Prometheus, Jaeger/Zipkin
US-11, US-12	Kafka + Audit/Notification + MongoDB + notification provider

6. Architecture Decision Log — v0.1

ADR	Decision	Status
ADR-001	Use microservice-owned data stores; no shared business database.	Accepted baseline
ADR-002	Use Keycloak + OAuth2/OIDC + JWT; enforce RBAC in downstream services.	Accepted baseline
ADR-003	Use Transactional Outbox + Debezium for OrderCreatedEvent.	Source-required
ADR-004	Use idempotent Payment consumer with processed_events.	Source-required
ADR-005	Use asynchronous PaymentSucceeded/PaymentFailed events back to Order Service.	PROPOSED
ADR-006	Keep v0.1 order lifecycle payment-focused; inventory/shipping deferred.	PROPOSED
ADR-007	Use a monorepo during portfolio implementation while preserving service boundaries.	PROPOSED

6.1 Recommended Implementation Order

8. Infrastructure skeleton: docker-compose with PostgreSQL, Kafka, Debezium, Redis, MongoDB, Keycloak, Prometheus and trace backend.
9. Catalog Service: schema + Flyway + JPA + EntityGraph + basic REST, then Redis/Redisson.
10. Order Service: order aggregate + state machine + outbox table and transaction.
11. Debezium → Kafka pipeline and OrderCreatedEvent contract.
12. Payment Service: consumer + processed_events + mock RestClient integration.
13. Payment result event → Order state transition.
14. Audit / Notification consumer + MongoDB.
15. Observability hardening and end-to-end trace propagation.
16. Spring Batch supplier import.
17. Integration/E2E tests and architecture documentation polish.

6.2 Open Decisions for the Next Revision

- Exact role model and which endpoints are public versus authenticated.
- Product availability semantics (simple status versus real inventory service).
- Payment result publishing reliability — direct Kafka producer or a second Transactional Outbox in Payment Service.
- Kafka retry / dead-letter topic strategy and maximum retry policy.
- Notification retry persistence and provider abstraction.
- Event schema governance (JSON Schema/Avro/Protobuf) and compatibility policy.
- Whether each service gets a separate PostgreSQL instance or separate database/schema in local development.

Next documentation set: After v0.1 is accepted, the next professional artifacts should be: API Specification (OpenAPI), Event Catalog, ADRs, Deployment/Runbook, Test Strategy, Threat Model, Sequence Diagrams for checkout/payment, and a Database Migration Guide.

Source Note

This package is based on the supplied project-plan text. The source establishes the technology stack, infrastructure requirements, four service responsibilities, Transactional Outbox + Debezium, Payment idempotency, Redis/Redisson caching, and observability. Any item explicitly labelled PROPOSED was introduced only to make the architecture implementable and should be reviewed as the project evolves.